Footballer Rating App

Software Requirements Specification

Ngong Samantha Kuo

Software Engineering

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the Footballer Rating App, a mobile application developed using React Native (frontend) and Node.js (backend).

1.2 Scope

The Footballer Rating App allows users to search for footballers, view their profiles and statistics, and submit community ratings and reviews. The app targets football fans who want a platform to rate and discuss players.

1.3 Definitions and Acronyms

- SRS - Software Requirements Specification
- API - Application Programming Interface
- UI - User Interface
- JWT - JSON Web Token
- REST - Representational State Transfer
- CRUD - Create, Read, Update, Delete

1.4 Overview

This document covers: Introduction, Overall Description, Functional Requirements, Non-Functional Requirements and Architecture Diagrams.

2. Overall Description

2.1 Product Perspective

The Footballer Rating App is a standalone Android mobile application. It consists of a React Native frontend and a Node.js REST API backend, with data stored in Firebase Firestore database.

2.2 Product Functions

The main functions of the application include:

- User registration and login
- Browse and search for footballers by name, club, nationality or position
- View footballer profiles with stats and community ratings
- Submit a rating (1-10 scale) and a written review for any footballer
- Edit or delete your own ratings and reviews
- View a leaderboard of the highest-rated footballers
- View a user profile showing all submitted reviews

2.3 User Classes

- Guest - Can browse footballer profiles and read reviews without logging in

- Registered User - Can submit, edit, and delete their own ratings and reviews

2.4 Operating Environment

- Platform: Android Mobile Devices
- Frontend: React Native using Expo
- Backend: Node.js with Express.js
- Database: Firebase Firestore
- Authentication: JWT-based token authentication

2.5 Assumptions and Dependencies

- Users have an Android device with an active internet connection
- Footballer data will either be seeded manually or retrieved from a public football API
- The backend server is deployed and accessible through the internet

3. Functional Requirements

3.1 User Authentication

- The app shall allow new users to register with a username, email, and password
- The app shall allow users to log in using their email and password
- The app shall issue a JWT token upon successful login for session management
- The app shall allow users to log out, clearing the stored token
- User passwords shall be hashed using bcrypt before being stored in the database

3.2 Footballer Browsing and Search

- The app shall display a scrollable list of footballers on the home screen
- The app shall allow users to search for footballers by name
- The app shall allow users to filter the list by position, nationality or club
- Each footballer listing shall display name, club, nationality, position, and average community rating

3.3 Footballer Profile

- The app shall display a detailed profile page for each footballer
- The profile shall show name, nationality, club, position, age, and photo
- The profile shall display the footballer's average rating calculated from all user ratings
- The profile shall list all user reviews in chronological order

3.4 Rating and Review System

- Registered users shall be able to submit a numeric rating from 1 to 10 for any footballer
- Registered users shall be able to include an optional written review with their rating
- A user may only submit one rating per footballer, submitting again will update the existing rating
- Registered users shall be able to edit their own submitted rating and review

- Registered users shall be able to delete their own rating and review
- Guest users shall not be able to submit ratings, instead they shall be prompted to register or log in

3.5 Leaderboard

- The app shall display a leaderboard of the top 20 highest-rated footballers
- The leaderboard shall update dynamically as new ratings are submitted

3.6 User Profile

- The app shall show a user profile page with the username and all reviews the user has submitted
- The user profile page shall show the total number of ratings submitted by the user

3.7 Error Handling

- The app shall display an error message if login credentials are incorrect
- The app shall notify users when network errors occur
- The app shall prevent duplicate ratings for the same footballer by the same user
- The app shall display a message when search results return no matches

4. Non-Functional Requirements

4.1 Performance

- API responses shall return within 5 seconds under normal network conditions
- The app shall handle at least 30-50 concurrent users using the free-tier backend deployment
- Lists of footballers and reviews should load using paginated API requests to reduce loading time.

4.2 Usability

- The interface shall be intuitive and usable without prior training
- The app shall maintain a consistent design language across all screens
- Loading indicators shall be displayed whenever the app is fetching data from the API
- Error messages shall be clear and written in plain, user-friendly language

4.3 Security

- All protected API endpoints shall validate the JWT token before processing requests
- Passwords shall never be stored or transmitted in plain text
- Users shall only be able to modify or delete their own ratings and reviews

4.4 Reliability

- The app shall handle network errors gracefully and display clear messages to the user
- The app shall not crash when receiving empty, null, or unexpected data from the API
- Failed API requests should allow the user to retry the operation.

4.5 Scalability

- The backend shall be built following RESTful principles to allow future feature additions
- The database schema shall support adding new footballers, statistics, or attributes without affecting existing data

4.6 User Interface Requirements

The application shall include the following main screens:

- Home screen: Displays a list of footballers
- Footballer profile screen: Displays player information and reviews
- Login and registration screens: Allows users to create accounts and authenticate
- Leaderboard screen: Displays the highest-rated footballers
- User profile screen: Displays the user’s information and reviews

The interface should follow standard mobile navigation patterns and support vertical scrolling for lists of players and reviews

5. System Architecture

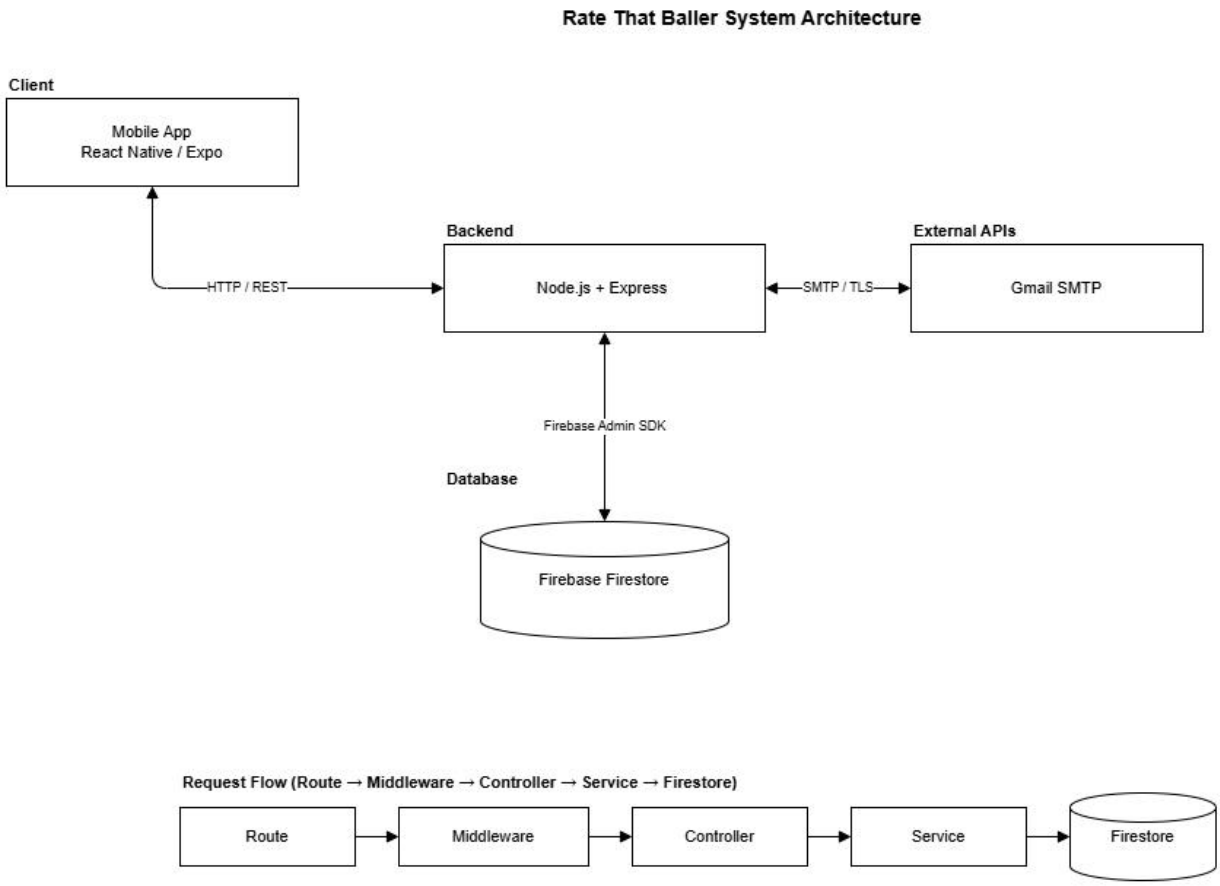
5.1 Technology Stack

- Frontend: React Native with Expo
- Backend: Node.js with Express.js
- Database: Firebase Firestore
- State Management: React Context API
- HTTP Client: Axios for API calls from the React Native app

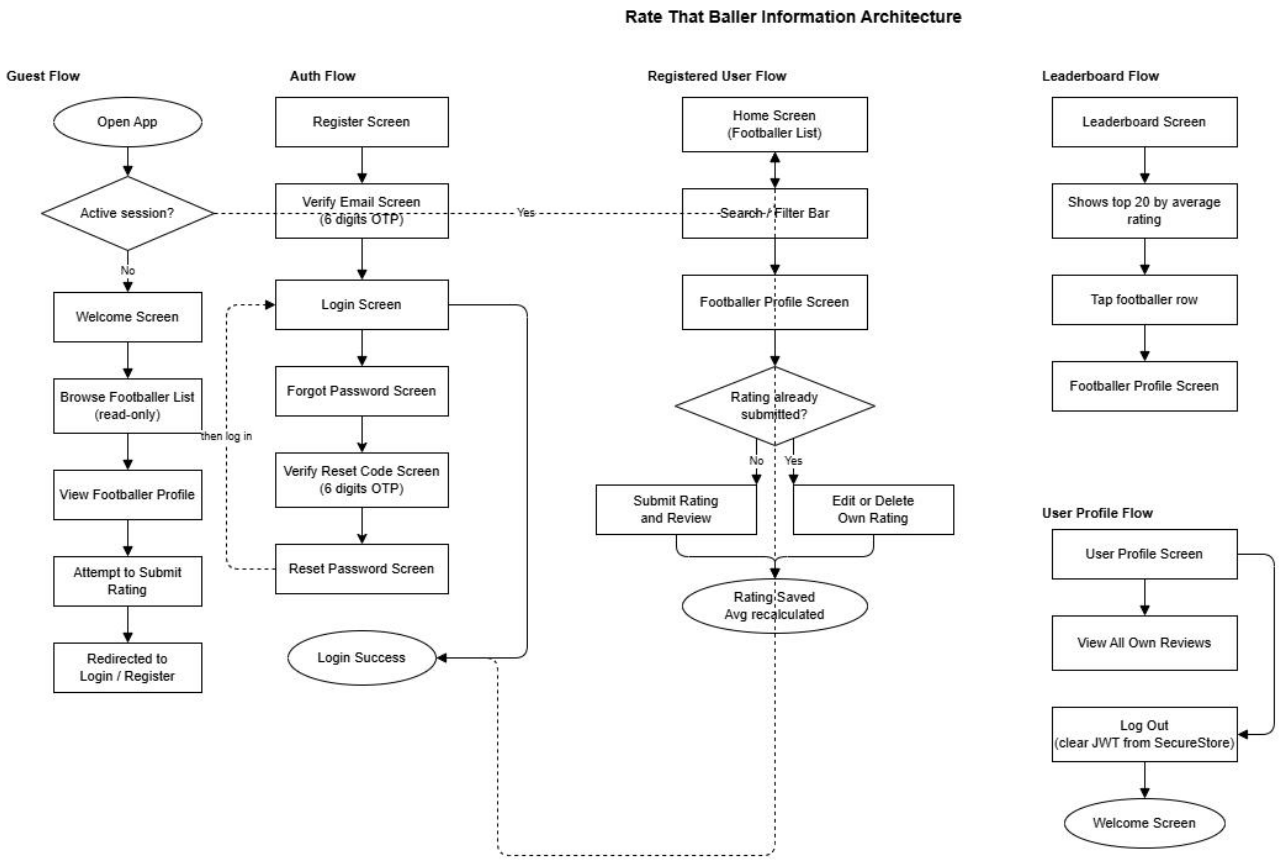
5.2 High-Level Architecture

The application follows a three-tier client-server architecture:

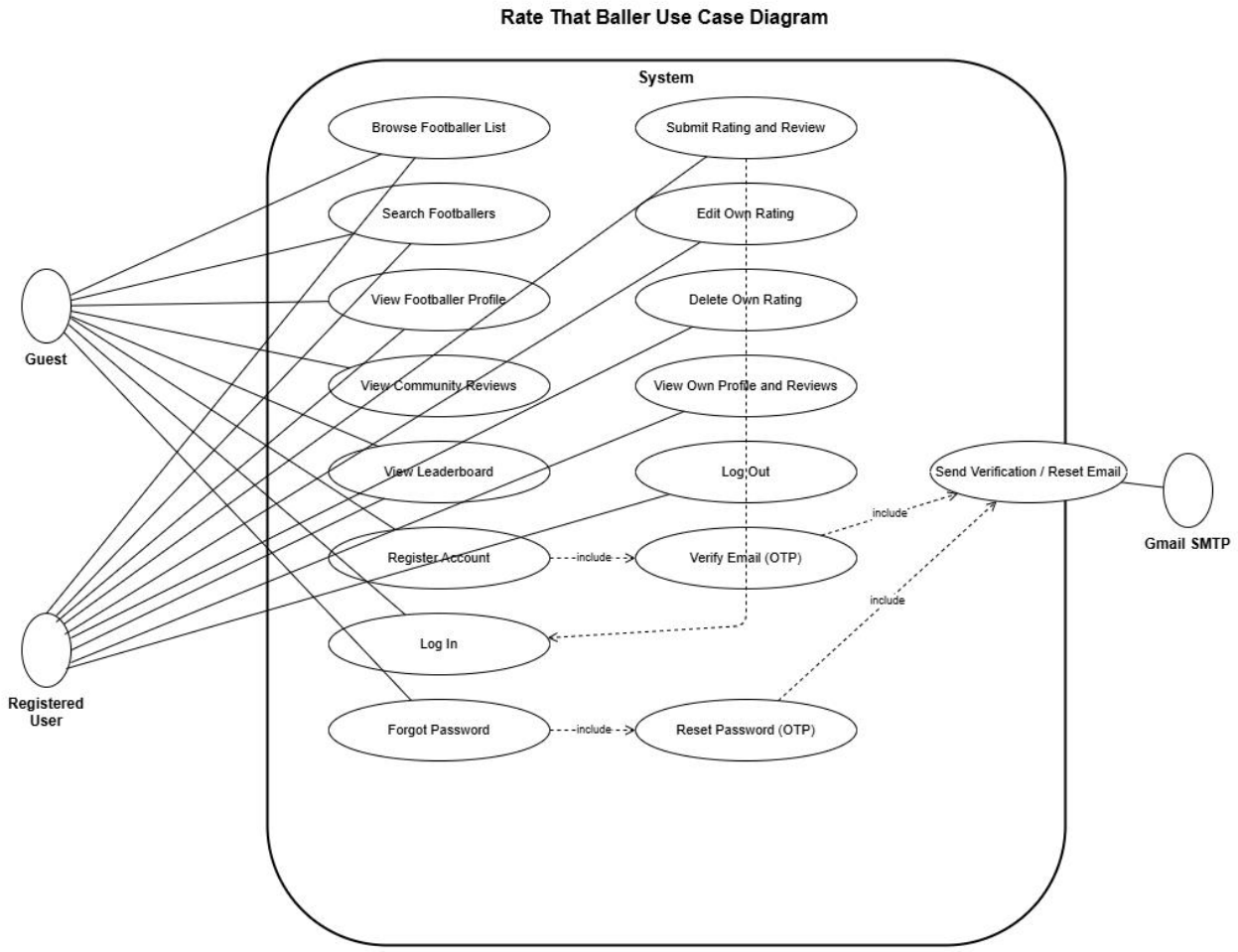
- Presentation Layer - React Native Expo app running on the user's Android device
- Application Layer - Node.js and Express REST API handling all business logic
- Data Layer - Firebase Firestore database storing users, footballers, and ratings



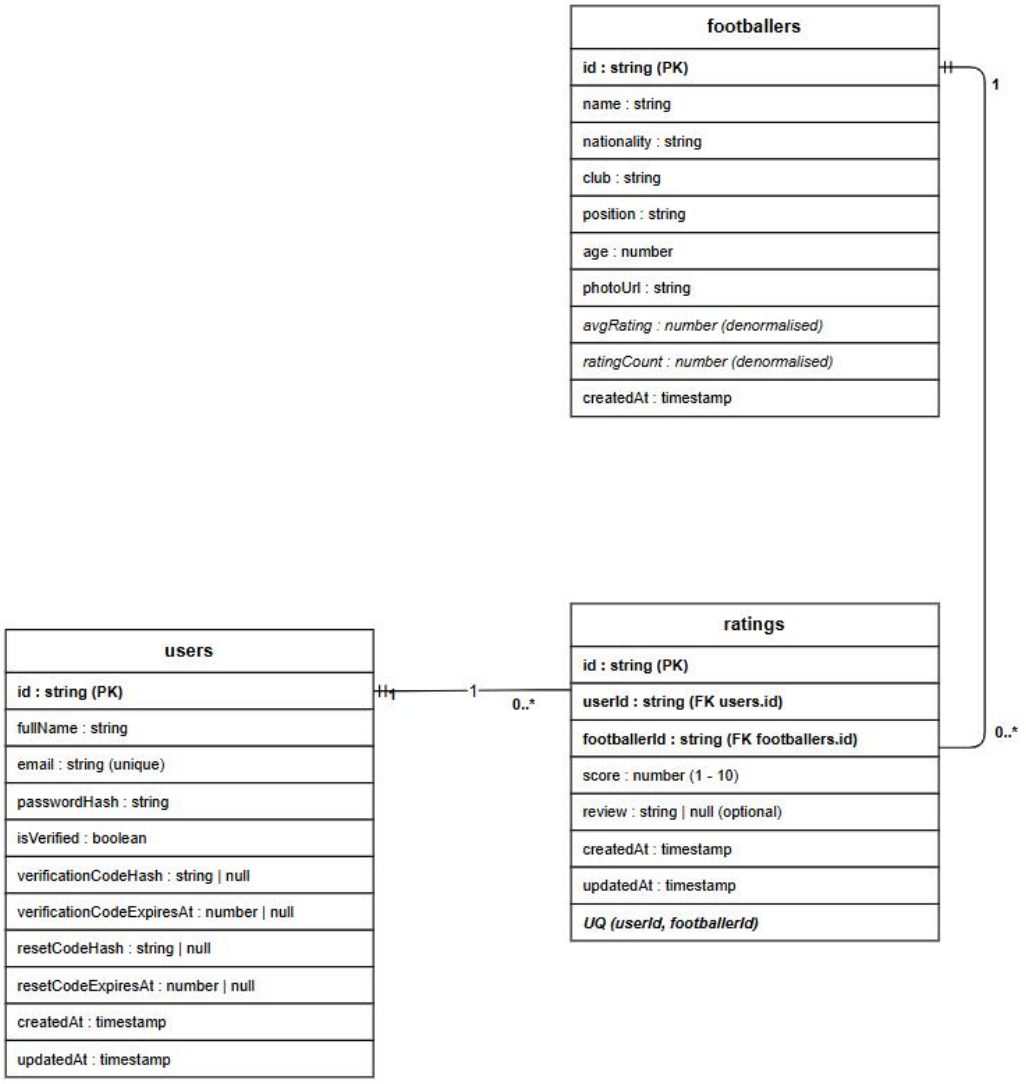
System Architecture Diagram



Information Architecture Diagram



Use Case Diagram



Entity Relationship Diagram (ERD)

5.3 Key API Endpoints

- POST /api/auth/register - Register a new user account
- POST /api/auth/login - Log in and receive a JWT
- GET /api/footballers - Get a list of all footballers
- GET /api/footballers/:id - Get a single footballer's full profile
- GET /api/footballers/search?q= - Search footballers by name or club or nationality
- POST /api/ratings - Submit a new rating (requires authentication)
- PUT /api/ratings/:id - Edit an existing rating (requires authentication)
- DELETE /api/ratings/:id - Delete a rating (requires authentication)
- GET /api/leaderboard - Get the top 20 rated footballers
- GET /api/users/:id - Get a user profile and their submitted reviews